RELATÓRIO TÉCNICO: MVP - SISTEMA DE AGENDAMENTO ONLINE

Autor: Yuri Rodrigues Alcantara
Data: Março de 2026
Disciplina: Fábrica de Software com Gestão Ágil

1. Visão Geral do Produto

O presente projeto consiste no desenvolvimento de um Produto Mínimo Viável (MVP) para um Sistema de Agendamento Online.

Problema Resolvido: A gestão ineficiente de agendas em pequenos negócios (clínicas, barbearias, consultorias), que resulta em horários ociosos, falhas de comunicação e perda de tempo produtivo com atendimento manual.

Proposta de Valor: Oferecer uma plataforma web intuitiva, responsiva e segura que centralize as marcações em um calendário interativo, garantindo autonomia para o cliente final e organização impecável para o administrador.

2. Gestão Ágil e Backlog do Produto

O ciclo de desenvolvimento foi executado em formato de Sprint única de 4 dias, simulando o ritmo de uma verdadeira Fábrica de Software. O acompanhamento das tarefas foi realizado via quadro Kanban (To Do, In Progress, Done).

O escopo foi delimitado pelas seguintes User Stories:

- [ALTA] Como administrador, quero acessar o sistema com login e senha protegidos para garantir a segurança dos dados.
- [ALTA] Como usuário, quero visualizar os horários disponíveis em formato de calendário para evitar conflitos de agenda.
- [ALTA] Como usuário, quero registrar um agendamento informando meu nome, serviço e data/hora.
- [ALTA] Como administrador, quero poder cancelar (excluir) um agendamento para manter o fluxo de trabalho atualizado.
- [MÉDIA] Como usuário, quero receber feedbacks visuais (notificações) ao realizar ações no sistema.

3. Arquitetura e Stack Tecnológica

A solução adotou uma arquitetura moderna baseada em Cliente-Servidor (Frontend e Backend desacoplados), permitindo escalabilidade e facilidade de manutenção.

Frontend (Interface do Usuário):
Framework: React estruturado com Vite (Single Page Application).
Bibliotecas: react-big-calendar e date-fns (renderização de calendários interativos), axios (consumo de APIs) e react-toastify (notificações e pop-ups de feedback).

Backend (API RESTful):
Framework: Python com FastAPI.
Persistência de Dados: Banco de dados relacional SQLite, orquestrado via SQLAlchemy (ORM).
Segurança e Autenticação: Implementação de JWT (JSON Web Tokens) e criptografia de senhas utilizando a biblioteca passlib com motor bcrypt (versão 3.2.2).

Garantia de Qualidade (QA) e Automação:
Testes de Integração: Pytest para validação das rotas e do banco de dados.
CI/CD Simulado e Cron Jobs: Utilização da biblioteca APScheduler para rodar scripts de teste em background automaticamente todos os dias às 22h00.

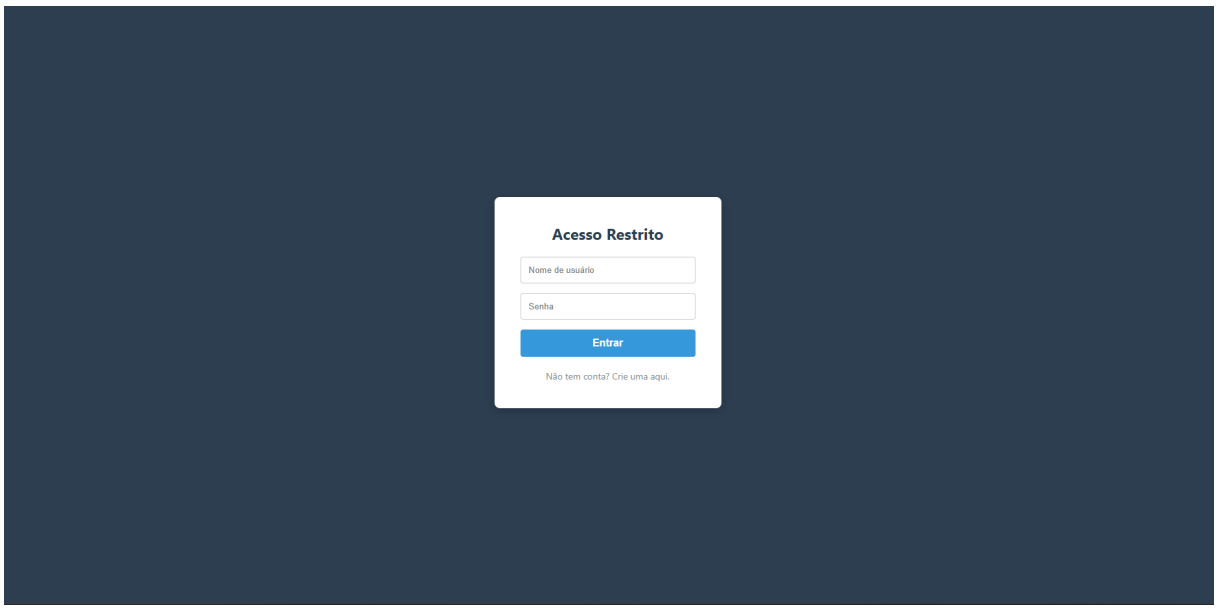
4. Funcionalidades Entregues e UI/UX

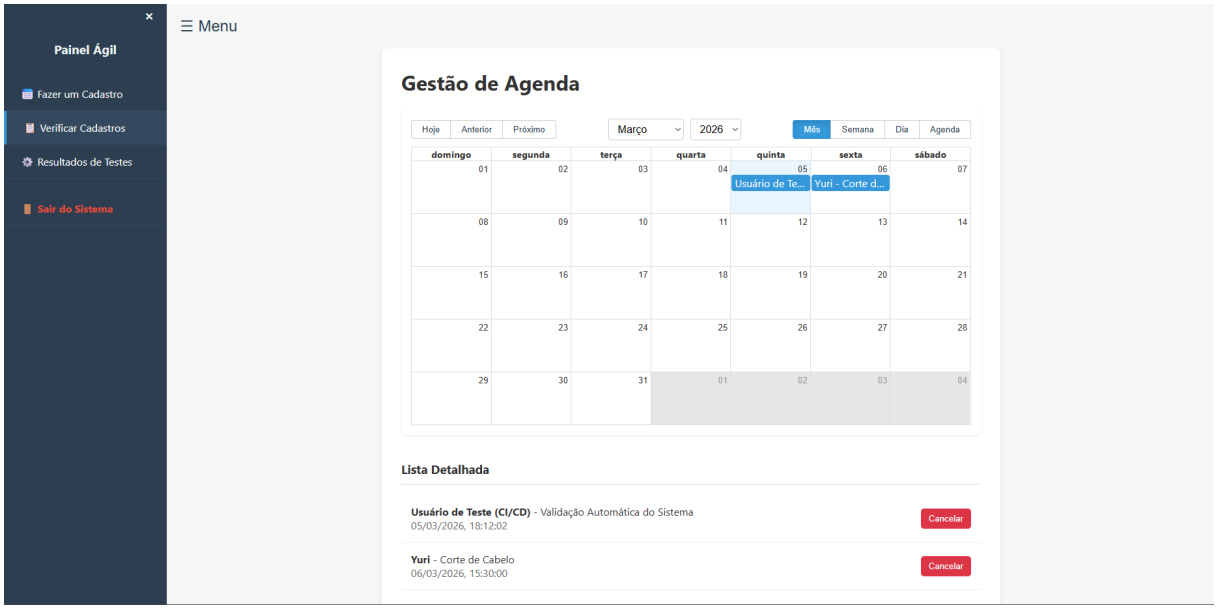
O sistema atendeu e superou os requisitos de CRUD estipulados no projeto, entregando as seguintes telas:

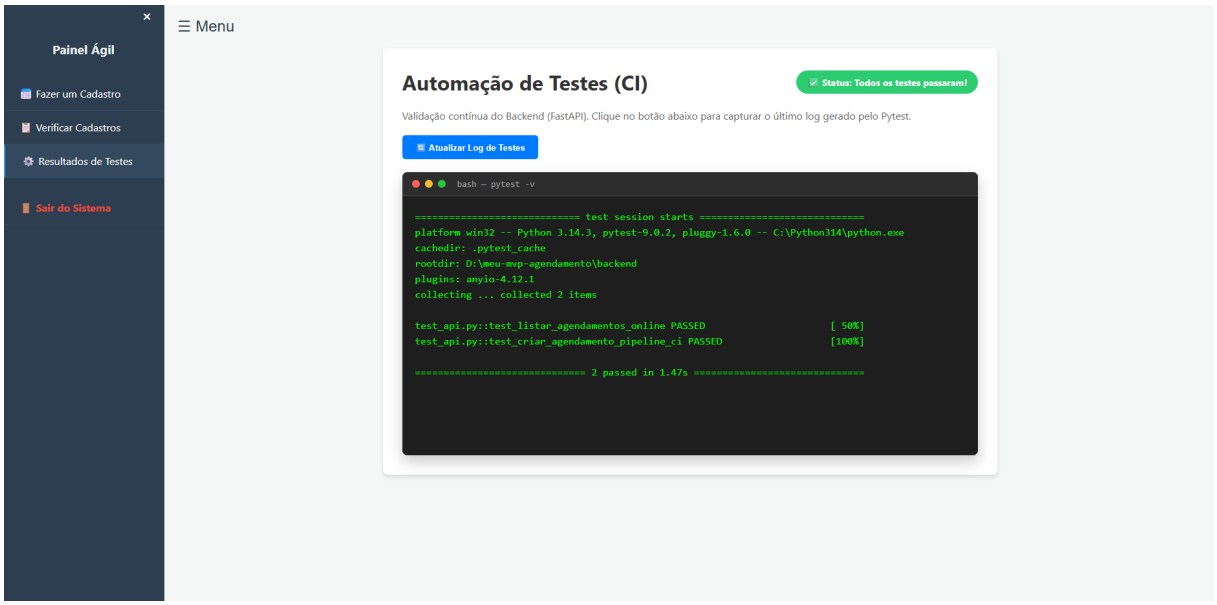
Módulo de Autenticação: Tela de login e cadastro de novos administradores, com senhas hasheadas no banco de dados. O sistema bloqueia rotas privadas caso o token JWT não seja identificado.

Dashboard de Agendamento: Formulário simplificado de marcação e visualização em grade de calendário mensal, semanal ou diária (estilo Teams/Google Calendar).

Sistema de Notificações (Toasts): Alertas coloridos e animados no canto superior da tela para confirmar ações de sucesso ou falha na comunicação com o servidor.







5. Diferenciais Técnicos (Bônus da Sprint)

Para elevar o nível de maturidade do software, foram implementadas rotinas de desenvolvimento de nível sênior:

Painel de Automação de Testes (CI): Uma interface dedicada no frontend que se comunica com uma rota exclusiva do backend (/api/run-tests). Ela permite ao administrador executar o Pytest sob demanda e visualizar o log de testes em tempo real em um terminal simulado na própria tela do sistema.

Testes de Integridade Agendados: O servidor possui um Cron Job interno configurado para executar uma bateria de testes automaticamente. Essa prática garante a saúde da aplicação sem necessidade de intervenção humana.

```
PS D:\meu-mvp-agendamento\backend> python -m uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['D:\\meu-mvp-agendamento\\backend']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [18532] using StatReload
INFO:     Started server process [20592]
INFO:     Waiting for application startup.
🚨 Agendador ativado! Testes programados para as 22:00.
INFO:     Application startup complete.
INFO:     127.0.0.1:60522 - "POST /register HTTP/1.1" 200 OK
INFO:     127.0.0.1:60522 - "POST /login HTTP/1.1" 200 OK
INFO:     127.0.0.1:60622 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57621 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:60622 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57621 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:62827 - "POST /login HTTP/1.1" 400 Bad Request
INFO:     127.0.0.1:52174 - "OPTIONS /register HTTP/1.1" 200 OK
INFO:     127.0.0.1:52174 - "POST /register HTTP/1.1" 200 OK
INFO:     127.0.0.1:52174 - "POST /login HTTP/1.1" 200 OK
INFO:     127.0.0.1:52174 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:54516 - "POST /login HTTP/1.1" 200 OK
INFO:     127.0.0.1:54516 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57877 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:61733 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:57877 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:61733 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:58228 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:58228 - "POST /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:58228 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:56384 - "POST /login HTTP/1.1" 200 OK
INFO:     127.0.0.1:56384 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:56384 - "OPTIONS /api/run-tests HTTP/1.1" 200 OK
INFO:     127.0.0.1:56384 - "GET /api/run-tests HTTP/1.1" 200 OK
INFO:     127.0.0.1:56384 - "GET /appointments/ HTTP/1.1" 200 OK
WARNING:  StatReload detected changes in 'test_api.py'. Reloading...
 INFO:     Shutting down
INFO:     Waiting for application shutdown.
🚨 Agendador desligado.
INFO:     Application shutdown complete.
INFO:     Finished server process [20592]
INFO:     Started server process [22212]
INFO:     Waiting for application startup.
🚨 Agendador ativado! Testes programados para as 22:00.
INFO:     Application startup complete.
INFO:     127.0.0.1:59305 - "OPTIONS /api/run-tests HTTP/1.1" 200 OK
INFO:     127.0.0.1:59305 - "GET /api/run-tests HTTP/1.1" 200 OK
INFO:     127.0.0.1:59305 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:59305 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:49468 - "OPTIONS /appointments/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:49468 - "DELETE /appointments/1 HTTP/1.1" 200 OK
INFO:     127.0.0.1:49468 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:49468 - "GET /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:54983 - "OPTIONS /appointments/ HTTP/1.1" 200 OK
INFO:     127.0.0.1:54983 - "POST /appointments/ HTTP/1.1" 200 OK
```

```
PS D:\meu-mvp-agendamento\frontend> npm run dev

> frontend@0.0.0 dev
> vite

17:54:36 [vite] (client) Re-optimizing dependencies because lockfile has changed

  VITE v7.3.1  ready in 1243 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
17:55:58 [vite] (client) ✨ new dependencies optimized: react-toastify
17:55:58 [vite] (client) ✨ optimized dependencies changed. reloading
```

6. Conclusão e Lições Aprendidas

O desenvolvimento deste MVP demonstrou que a combinação de React com FastAPI proporciona extrema velocidade de entrega, alinhando-se perfeitamente aos princípios das metodologias ágeis.

Retrospectiva:

O que funcionou bem: A modularização do código e a adoção rápida de bibliotecas prontas (como o React Big Calendar) economizaram dezenas de horas de desenvolvimento, mantendo o foco no valor de negócio.

Oportunidades de Melhoria (Próximas Sprints): Expansão do sistema para suportar perfis distintos de usuários (Admin vs. Cliente Padrão) e envio automatizado de lembretes por e-mail/WhatsApp aos clientes agendados.